Atividade Integradora — Java e POO

Sistema Acadêmico Simplificado | Respostas teóricas e análise da implementação

Parte I — Questões teóricas

Bloco A — Fundamentos de POO

1. Classe e objeto. Classe é o molde que define dados e operações; objeto é uma instância concreta desse molde. No sistema, Curso é uma classe; new Curso("Java", 200.00) cria um objeto Curso.

2. Atributos e métodos; estado e comportamento. Atributos guardam dados e compõem o estado; métodos representam ações e compõem o comportamento. Em Aluno, nome é um atributo e adicionarCurso é um método.

3. new e referência. new instancia um objeto e reserva espaço para ele. Declarar Curso curso; cria apenas uma variável de referência; curso = new Curso(...) cria o objeto e faz a referência apontar para ele.

4. Encapsulamento. Encapsulamento protege os dados internos e controla como são consultados ou alterados, por exemplo, com atributos private e métodos públicos. Atributos públicos permitem alterações sem validação, quebrando invariantes como preço positivo ou nome preenchido.

5. Construtor e this. O construtor configura o objeto no momento da criação e pode validar seus valores. this referencia o objeto atual e distingue atributo de parâmetro, como em this.nome = nome.

Bloco B — Relacionamentos e herança

6. Associação, agregação e composição. Associação é uma ligação entre objetos. Agregação é uma relação todo-parte em que a parte pode existir independentemente. Composição é uma relação mais forte, em que o ciclo de vida da parte depende do todo. Aluno e Curso neste exercício são associados; cursos não são destruídos quando o aluno deixa de existir.

7. ArrayList e vetor. ArrayList é uma coleção redimensionável que permite adicionar e remover elementos durante a execução. É útil quando a quantidade de cursos de um aluno varia. Um vetor tradicional tem tamanho fixado quando criado.

8. Superclasse, subclasse, extends e super. Superclasse é a classe mais geral; subclasse especializa ou reaproveita seus membros. extends declara herança entre classes. super acessa construtor ou membro da superclasse.

9. Sobrecarga e sobrescrita. Sobrecarga define métodos de mesmo nome com listas de parâmetros diferentes na mesma classe, resolvidos pela assinatura. Sobrescrita redefine na subclasse um método herdado com assinatura compatível; @Override ajuda a detectar erros.

10. Polimorfismo. Polimorfismo permite tratar objetos diferentes por um tipo comum e obter comportamentos específicos em tempo de execução. Uma referência Desconto pode apontar para SemDesconto ou DescontoEstudante; a chamada a aplicar executa a implementação do objeto real.

Bloco C — Interfaces e projeto

11. Interface; implements e extends. Interface define um contrato de operações sem exigir uma implementação concreta. Uma classe usa implements para cumprir uma interface; uma classe usa extends para herdar de outra classe. Interfaces também podem estender outras interfaces.

12. Herança e composição. Herança expressa uma relação “é um” e compartilha especialização. Composição expressa “tem um” e delega trabalho a objetos colaboradores. Composição costuma

ser preferível quando se quer trocar comportamentos, reduzir dependência de uma hierarquia rígida ou combinar capacidades.

13. Baixo acoplamento e injeção por construtor. Baixo acoplamento significa que componentes conhecem poucos detalhes uns dos outros. Receber dependências no construtor, como Desconto e RegistradorPagamento, permite trocar implementações sem reescrever o serviço e torna explícito o que ele precisa.

14. SRP, OCP e DIP. SRP: uma classe deve ter uma responsabilidade principal; por exemplo, ServiçoMatricula coordena matrícula, enquanto AdaptadorPagamento trata integração legada. OCP: componentes devem aceitar extensões sem alterações frequentes; uma nova política de desconto pode implementar Desconto. DIP: módulos de alto nível dependem de abstrações; ServiçoMatricula usa RegistradorPagamento em vez de depender diretamente do sistema legado.

Bloco D — Padrões de projeto

15. Padrão de projeto. É uma solução recorrente e nomeada para um problema de projeto de software. Deve ser aplicado quando o problema pede essa estrutura; uso indiscriminado acrescenta classes e complexidade sem benefício.

16. Singleton. Singleton restringe uma classe a uma única instância acessível globalmente. O construtor privado impede instanciação externa; getInstancia() devolve a instância estática compartilhada. No exemplo, a instância é criada uma vez no campo INSTANCIA.

17. Factory Method. Factory Method delega a criação de um produto para um método sobrescrito por criadores concretos. O criador abstrato define criar() e o fluxo de emissão; CriadorTexto e CriadorResumido escolhem o comprovante concreto.

18. Strategy. Strategy encapsula cada regra de desconto atrás da mesma interface. ServiçoMatricula chama aplicar sem conhecer se a regra é comum, estudantil ou promocional; basta injetar a estratégia desejada.

19. Observer. Observer mantém uma lista de interessados e avisa todos quando ocorre um evento. ServiçoMatricula produz o evento de matrícula confirmada; AvisoEmail, Auditoria e outros observadores recebem a mensagem e reagem independentemente.

20. Adapter e Facade. Adapter converte uma interface ou formato para outro: AdaptadorPagamento transforma reais em centavos para o sistema legado. Facade oferece uma operação simples sobre vários componentes: FachadaAcademica coordena matrícula e emissão do comprovante.

Parte II — Perguntas de análise da implementação

1. Qual classe implementa Singleton e quais instruções controlam a instância? Configuracao. A classe é final, guarda a instância em private static final INSTANCIA, tem construtor privado e fornece acesso por getInstancia().

2. Onde aparecem encapsulamento e validação? Os atributos de Aluno e Curso são private final. Os construtores verificam nome, título e preço; adicionarCurso valida o curso; ServiçoMatricula e AdaptadorPagamento validam dependências e valores.

3. Qual é a relação entre Aluno e Curso? Associação: Aluno mantém uma coleção de cursos, mas Curso pode existir sem aquele aluno. Não é composição, pois o ciclo de vida do curso é independente.

4. Onde foi utilizada uma coleção e com qual objetivo? ArrayList é usado em Aluno para armazenar cursos e em ServiçoMatricula para armazenar observadores. Ambas as listas podem crescer conforme novos itens são adicionados.

5. Qual é o papel da interface Desconto? Definir o contrato aplicar(valor) para diferentes estratégias. O serviço depende desse contrato e não de uma política específica.

6. Como acrescentar uma política de 20% sem alterar o serviço? Criar uma classe que implemente Desconto e retorne valor * 0.80, como DescontoPromocional20 no código fornecido, e injetá-la na criação de ServiçoMatricula.

7. Quem produz o evento e quem recebe as notificações? ServiçoMatricula produz a notificação após registrar o pagamento e adicionar o curso ao aluno. Os observadores inscritos, como AvisoEmail e Auditoria, recebem a mensagem pelo método atualizar.

8. Por que o adaptador converte reais em centavos? Porque o sistema legado espera um inteiro em centavos, enquanto a interface atual recebe reais. O adaptador faz a conversão e arredonda o resultado para o centavo mais próximo.

9. Qual método representa Factory Method? CriadorComprovante.criar(), declarado abstrato e implementado pelos criadores concretos para produzir ComprovanteTexto ou ComprovanteResumido.

10. O que FachadaAcademica simplifica para Main? Esconde a sequência de coordenação: chamar o serviço de matrícula, obter o total e pedir a emissão do comprovante. Main usa uma única operação de alto nível, matricularEEmitir.

Arquivos da implementação: o código Java correspondente foi preparado em Main.java. Para execução, salve-o em src/Main.java e use os comandos indicados no enunciado. Este ambiente não possui javac/JDK, portanto a compilação e a captura real da execução não puderam ser feitas aqui. A saída esperada está no arquivo de saída de referência.